Epicycles - A Bidirectional DSL

Alexander Wood

University of Bristol

1980-01-01

Outline

1. Introduction

2. Background

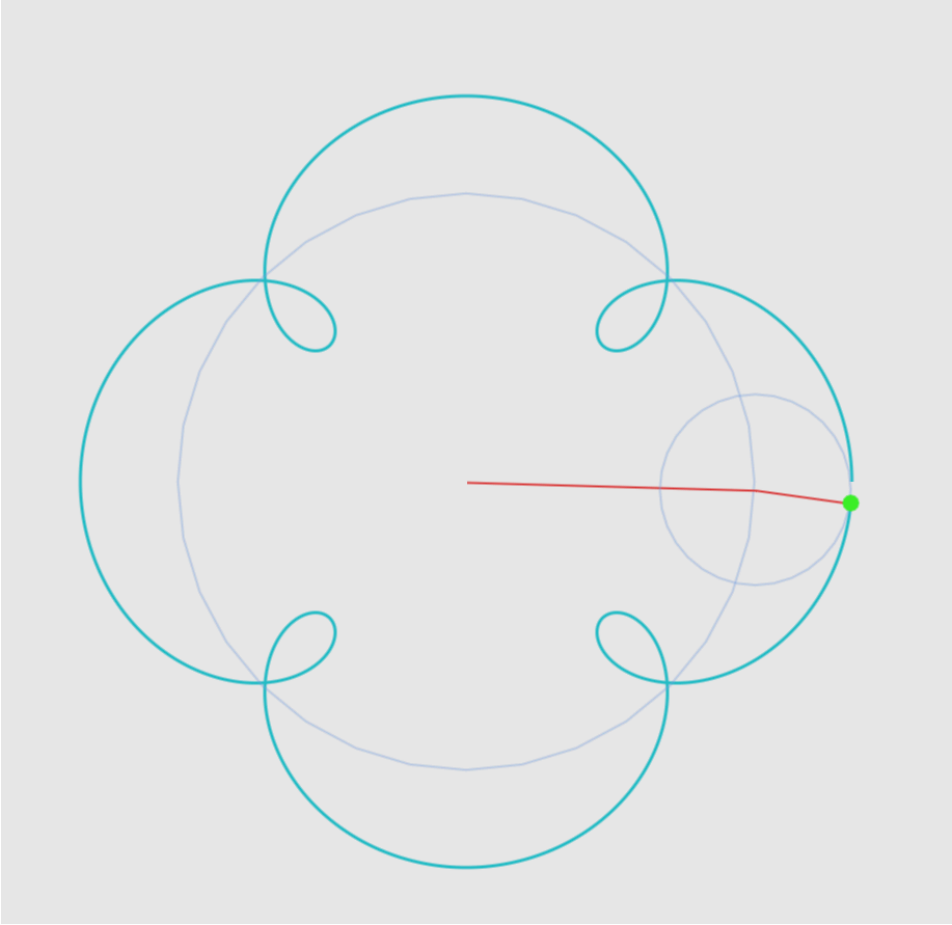
3. My DSL

4. Evaluation

5. References

1.1 Objectives

- Make a **bidirectional** DSL for making geometric art
  - *TidalCycles* for graphics
  - Edit the output and get new code!



Outline

1. Introduction

2. Background

3. My DSL

4. Evaluation

5. References

2. Background

- Updates are often ambiguous
  - e.g. “move this point right by 5 pixels”
  - How does the code change?
  - *View-Update Problem*
- Existing solutions push ambiguity to the user
  - e.g. *Sketch-n-Sketch*[1] asks the user to choose between multiple options
  - Updates feel clunky

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…

2.1 Possible Solutions

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…
- … don't handle sharing well

2.1 Possible Solutions

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…
- … don't handle sharing well

2.1.2 🧮 Heuristics

- Handles sharing better than lenses

2.1 Possible Solutions

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…
- … don't handle sharing well

2.1.2 🧮 Heuristics

- Handles sharing better than lenses
- Can feel unpredictable to the user

2.1 Possible Solutions

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…
- … don't handle sharing well

2.1.2 🧮 Heuristics

- Handles sharing better than lenses
- Can feel unpredictable to the user

2.1.3 ⛰️ Automatic Differentiation

- Use gradients to find the best update

2.1 Possible Solutions

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…
- … don't handle sharing well

2.1.2 🧮 Heuristics

- Handles sharing better than lenses
- Can feel unpredictable to the user

2.1.3 ⛰️ Automatic Differentiation

- Use gradients to find the best update
- Very domain specific, not very PL-y

2.1 Possible Solutions

2.1 Possible Solutions

2.1.1 🔎 Lenses

- Well-studied, easy to reason about…
- … don't handle sharing well

2.1.2 🧮 Heuristics

- Handles sharing better than lenses
- Can feel unpredictable to the user

2.1.3 ⛰️ Automatic Differentiation

- Use gradients to find the best update
- Very domain specific, not very PL-y

2.1 Possible Solutions

2.1.4 🔗 Domain Restriction

- Can we remove the ambiguity altogether…?

2.2 Domain Restriction

- Design a system where updates are (mostly) unambiguous by design
- View-Update problem comes from View being too separate from the Source
- Make the view and source the same thing!
- Affects syntax *and* UI design

2.2 Domain Restriction

- Design a system where updates are (mostly) unambiguous by design
- View-Update problem comes from View being too separate from the Source
- Make the view and source the same thing!
- Affects syntax *and* UI design

Make DSL for *Fourier Series*

- Each term is a circle
- View $\longleftrightarrow$ Source, so updates are unambiguous

Outline

1. Introduction

2. Background

3. My DSL

4. Evaluation

5. References

3.1 Syntax

$$
\begin{array}{rcll}
\text{Expr} & e & ::= & x & \textit{variable} \\
 & & | & n \in \mathbb{Q} & \textit{literal} \\
 & & | & e_1 + e_2 & \textit{add} \\
 & & | & e_1 * e_2 & \textit{mul} \\
 & & | & e_1^{-1} & \textit{recip} \\
 & & | & \text{let } x = e_1 \text{ in } o & \textit{let} \\
\\
\text{Curve} & c & ::= & \text{epicycle}\left(e_r, e_\omega, e_\varphi\right) & \textit{epicycle} \\
 & & | & c_1 \oplus c_2 & \textit{concat} \\
\\
\text{CurveOrExpr} & o & ::= & c & \textit{curve} \\
 & & | & e & \textit{expr}
\end{array}
$$

3.1 Syntax

An epicycle term represents a spinning circle with radius e_r, rotation frequency e_ω, and phase shift e_φ.

Concatenating two curves c_1 and c_2 with $\oplus$: draw c_1 with c_2 centered at the trace of c_1

Let bindings *only* bind numeric values!

3.2 Statics

$$\eta ::= \mathbb{Z} \mid \mathbb{Q} \qquad \tau ::= \eta \mid \text{Curve}$$

$$\frac{\Gamma \vdash e_1 : \eta \qquad \Gamma, x : \eta \vdash o : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } o : \tau}\ \textsc{T-Let}$$

$$\frac{n \in \mathbb{Z}}{\Gamma \vdash n : \mathbb{Z}}\ \textsc{T-LitInt} \qquad \frac{n \in \mathbb{Q}}{\Gamma \vdash n : \mathbb{Q}}\ \textsc{T-LitRat}$$

$$\frac{\Gamma \vdash e_r : \eta \qquad \Gamma \vdash e_\omega : \mathbb{Z} \qquad \Gamma \vdash e_\varphi : \eta}{\Gamma \vdash \text{epicycle}\left(e_r, e_\omega, e_\varphi\right) : \text{Curve}}\ \textsc{T-Epicycle}$$

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{T-Var} \qquad \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 + e_2 : \tau}\ \textsc{T-Add}$$

$$\frac{\Gamma \vdash c_1 : \text{Curve} \qquad \Gamma \vdash c_2 : \text{Curve}}{\Gamma \vdash c_1 \oplus c_2 : \text{Curve}}\ \textsc{T-Concat}$$

3.3 Implementation

- Deeply Embedded DSL in Haskell
- Use *Embedding by Unembedding*[2] for HOAS frontend, FOAS backend
- *Trees that Grow* for type safety

```
curve :: (CurveExpr e) => e SCurve
curve =
  let_ 60 $ \a ->
    epicycle a 1 0
      <> epicycle (a * recip 3) 3 0
```

3.4 Forward Semantics

$$\frac{x : \tau \in \sigma}{\sigma, t \vdash x : \tau \Downarrow \sigma(x)} \text{E-Var}$$

$$\frac{\sigma, t \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma, t \vdash e_2 : \eta \Downarrow v_2}{\sigma, t \vdash e_1 + e_2 : \eta \Downarrow v_1 + v_2} \text{E-Add}$$

$$\frac{\sigma, t \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma[x \mapsto v_1], t \vdash o : \tau \Downarrow v}{\sigma, t \vdash \text{let } x = e_1 \text{ in } o : \tau \Downarrow v} \text{E-Let}$$

$$\frac{\sigma, t \vdash e_r : \mathbb{Q} \Downarrow v_r \qquad \sigma, t \vdash e_\omega : \mathbb{Z} \Downarrow v_\omega \qquad \sigma, t \vdash e_\varphi : \mathbb{Q} \Downarrow v_\varphi}{\sigma, t \vdash \text{epicycle}\left(e_r, e_\omega, e_\varphi\right) : \text{Curve} \Downarrow v_r \cdot e^{i\left(v_\omega \cdot t + v_\varphi\right)}} \text{E-Epicycle}$$

$$\frac{\sigma, t \vdash c_1 : \text{Curve} \Downarrow v_1 \qquad \sigma, t \vdash c_2 : \text{Curve} \Downarrow v_2}{\sigma, t \vdash c_1 \oplus c_2 : \text{Curve} \Downarrow v_1 + v_2} \text{E-Concat}$$

3.5 Backward Semantics

We have removed amguity, how do we update?

3.5 Backward Semantics

We have removed amguity, how do we update? Implement and compare two different backward semantics:

- 🔀 Sketch-n-Sketch style + heuristics[1]
- 🔺 Delta-based with program synthesis[3]

3.6 Sketch-n-Sketch Style Heuristics

A slightly simplified version of Sketch-n-Sketch's semantics[4]:

- Non-deterministic!
- Updates $x + y$ by fixing either x or y and updating the other to match
- Evaluate all candidates with a cost function

[1] e.g. MATLAB, Maple, etc

3.6 Sketch-n-Sketch Style Heuristics

A slightly simplified version of Sketch-n-Sketch's semantics[4]:

- Non-deterministic!
- Updates $x + y$ by fixing either x or y and updating the other to match
- Evaluate all candidates with a cost function
- Intuitive for the user... but violates correctness 🙁

[1] e.g. MATLAB, Maple, etc

3.6 Sketch-n-Sketch Style Heuristics

A slightly simplified version of Sketch-n-Sketch's semantics[4]:

- Non-deterministic!
- Updates $x + y$ by fixing either x or y and updating the other to match
- Evaluate all candidates with a cost function
- Intuitive for the user... but violates correctness 🙁
- In production, requires outsourcing to Computer Algebra System 😢[1]

Rule	Holds?
GetPut	✅
PutGet	🥲
PutPut	🪦

[1] e.g. MATLAB, Maple, etc

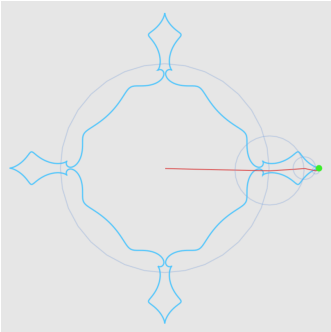
3.7 ▲ Delta-Based with Program Synthesis

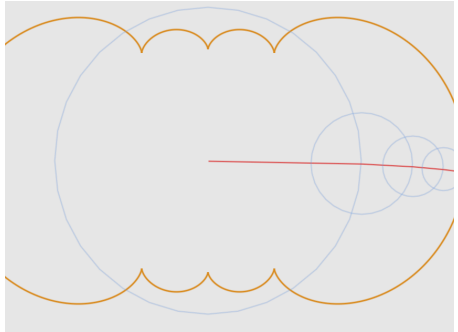
- Based on [3] – use a separate delta language to represent updates
- Turn deltas into program rewrites when conflicts arise
  - $\text{let } r = 10 \text{ in } r * r \mapsto \text{let } r = 10 \text{ in } (r + 1.1) * r$
- Deterministic - left biased
- Much more well-behaved, correctness guaranteed

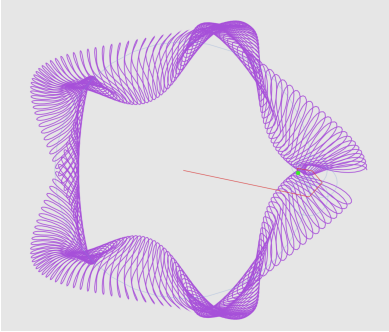
Rule	Holds?
GetPut	✅
PutGet	✅
PutPut	✅

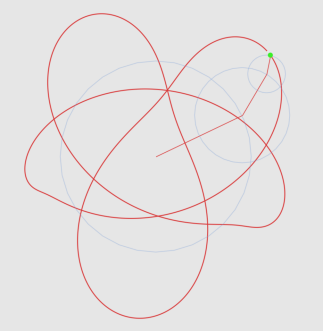
3.8 Expressivity

- Changing a few parameters creates variety of interesting patterns
- **Discrete Fourier Transform** lets us represent any (continuous) curve as a sum of epicycles!
- Can approximate text, images, etc









Outline

1. Introduction

2. Background

3. My DSL

4. Evaluation

5. References

4.1 The Tradeoffs

4.1.1 🔀 Sketch-n-Sketch

🧑‍🎨 🙂 Intuitive edits (usually)

🧑‍💻 🙁 $\approx 6\%$ PutPut failure, common PutGet failure

- let $r = 10$ in $r * r \mapsto$ 💥

4.1.2 🔺 Delta-Based

🧑‍🎨 🥲 Mutates AST shape

🧑‍💻 🙂 Correctness guaranteed

- Bounded AST growth (max $\approx 1.4\times$)
- Much harder to prove formally!

4.2 Conclusions & Future Work

- ? Mechanically verified properties in Lean (Delta proofs assisted by LLM).
- ? Proved DSL normal form & delta growth reaches a fixed point.
- ? **Conclusion:** Delta semantics are superior, but good UX isn't always correct.
- ? **Future Work:** Move away from embedding, new target domain, allow "unlocks" for shared vars

4.3 Thank You!

- 🔗 = 🗃️ ✅
- 🔀 = 👮‍♂️ 🙂 + 👨‍💻 🙁
- 🔺 = 👮‍♂️ 🥲 + 👨‍💻 🙂

Outline

1. Introduction

2. Background

3. My DSL

4. Evaluation

5. References

5. References

[1] B. Hempel, J. Lubin, and R. Chugh, "Sketch-n-Sketch: Output-Directed Programming for SVG," in *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*, New Orleans LA USA: ACM, Oct. 2019, pp. 281–292. doi: 10.1145/3332165.3347925.

[2] K. Matsuda, S. Frohlich, M. Wang, and N. Wu, "Embedding by Unembedding," *Proceedings of the ACM on Programming Languages*, vol. 7, no. ICFP, pp. 1–47, Aug. 2023, doi: 10.1145/3607830.

[3] X. Zhang, R. Xie, G. Guo, X. He, T. Zan, and Z. Hu, "Fusing Direct Manipulations into Functional Programs," *Proceedings of the ACM on Programming Languages*, vol. 8, no. POPL, pp. 1211–1238, Jan. 2024, doi: 10.1145/3632883.

5. References

[4] M. Mayer, V. Kuncak, and R. Chugh, "Bidirectional evaluation with direct manipulation," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–28, Oct. 2018, doi: 10.1145/3276497.